

Auditoría Completa — Wallet Control

Todo lo que conviene revisar antes de compartir la app con tus amigos

Generado el 27 de julio de 2026 · Revisión de código (4 análisis en paralelo) + pruebas en vivo en tablet Android

39

HALLAZGOS TOTALES

11

ALTA PRIORIDAD

21

MEDIA PRIORIDAD

7

BAJA PRIORIDAD

Antes que nada: el hallazgo #5 de la sección Alta (variables de entorno de Supabase) conviene revisarlo **ya mismo**, incluso antes de tocar código — puede hacer que el próximo build de "SUBIR PROYECTO" (EAS) se abra y se cierre solo en el celular de tus amigos, sin ni siquiera llegar a la pantalla de login.

Prioridad Alta — arreglar antes de compartir

Bugs que pierden datos del usuario, rompen una función completa, o pueden hacer crashear el build.

1. Los ingresos no se pueden editar ni eliminar

`lib/storage.ts`

Existen `deleteExpense/updateExpense` para gastos, pero no hay equivalente para ingresos — la única función es `addIncomes`.

Escenario: alguien anota su sueldo con un cero de más por error. Ese ingreso queda pegado para siempre en todos los totales del mes — no hay ningún botón, en ninguna pantalla, para corregirlo o borrarlo.

2. Los datos del modo anónimo no se pueden pasar a una cuenta real

`lib/storage.ts` · `app/onboarding.tsx` · `app/(tabs)/perfil.tsx`

Todo se guarda con una "llave" que depende del id del perfil (o 'anon' si no hay cuenta). Al crear una cuenta real se genera un id nuevo, y nada copia lo que ya existía bajo anon.

Escenario: justo el camino que la app recomienda a un usuario nuevo desconfiado — "prueba en modo anónimo primero" — se vuelve una trampa: si después decide crear cuenta para desbloquear Finando IA, todas sus tarjetas, gastos y metas parecen haberse borrado (siguen en el dispositivo, pero ya inalcanzables desde la app).

3. El recordatorio de un gasto/préstamo ignora la fecha que elegiste y siempre usa "hoy"

`lib/notifications.ts` · `resumen.tsx` · `CategoryDetailModal.tsx`

`scheduleRecurringReminder` siempre se llama con `new Date()`, nunca con la fecha de vencimiento que el usuario eligió en el calendario.

Escenario: marcas "ARRIENDO" como recurrente y eliges que vence el día 30. La notificación real llega el día que lo creaste (hoy), no el 30 — nunca cerca de la fecha real. El campo de fecha de vencimiento es puramente decorativo para efectos de notificación.

4. Las notificaciones de préstamos quedan huérfanas o desactualizadas

`components/CardFormModal.tsx` · `app/(tabs)/tarjetas.tsx` · `lib/storage.ts`

Tres huecos relacionados en el mismo mecanismo: (a) eliminar una tarjeta/préstamo nunca cancela su notificación programada; (b) apagar el switch "Notificarme" en un préstamo que ya tenía recordatorio no lo cancela; (c) el switch siempre vuelve a "apagado" al editar, así que cambiar solo la fecha de vencimiento no reprograma nada.

Escenario: creas un préstamo con recordatorio para el día 15, luego lo pagas y lo eliminas (o solo cambias la fecha a fin de mes). El celular igual avisa el día 15 original sobre una deuda que ya no existe o que ya cambió de fecha.

5. Las variables de Supabase pueden hacer crashear el build de EAS (revisar antes del próximo "SUBIR PROYECTO")

`lib/supabase.ts`

`process.env.EXPO_PUBLIC_SUPABASE_URL!` y su `ANON_KEY` se usan con `!"` (asumiendo que existen) sin ningún control. No hay evidencia en `eas.json` de que esas variables estén registradas como secretos/env de EAS — solo existen en tu `.env local`, que está en `.gitignore` y por lo tanto no viaja al build.

Escenario real: si esas dos variables no están cargadas en el perfil de build de EAS, la app crashea al abrir — antes incluso de mostrar la pantalla de login — para cada amigo que la instale.

✅ Se puede confirmar en 30 segundos sin tocar código: correr `eas env:list` (o revisarlo en el dashboard de EAS) y verificar que ambas variables estén ahí para el perfil "preview".

6. Finando IA se queda "Analizando..." para siempre si falla la red

`lib/claude.ts`

El fetch a la función de Claude no tiene timeout ni `AbortController`; `loading` solo se apaga cuando la promesa se resuelve.

Escenario: a un amigo con datos móviles inestables se le congela el spinner para siempre, sin mensaje de error y sin botón para cancelar — solo cerrando la app a la fuerza se recupera.

7. Finando IA reconoce la tarjeta mencionada pero el gasto se guarda sin ninguna cuenta asignada

`lib/expenseParser.ts` · `constants/systemPrompt.ts`

Claude devuelve `cardName` cuando el usuario nombra una cuenta ("pagué con Nequi") y por eso no pregunta después — pero `buildExpenses` nunca lee ese campo; el gasto siempre se guarda con `cardId: undefined`.

Escenario: "pagué el mercado \$80.000 con Nequi" → el gasto se registra sin tarjeta, y como la app tampoco pregunta después (porque Claude ya dijo que no hacía falta), nadie se entera hasta que revisa Tarjetas a mano.

8. El modo demo de Finando trunca montos grandes (pierde un millón de pesos)

`lib/claude.ts`

La expresión regular del modo demo (`([\d]{2,7}[.,]?\d{0,3})`) solo reconoce un grupo de separador de miles. "\$1.800.000" se interpreta como 800000, perdiendo el "1." completo.

Escenario: se activa solo cuando no hay sesión válida (ver #14 abajo), pero cualquier mensaje tipo "arriendo \$1.800.000" en ese modo guarda \$800.000 sin ningún aviso de que el monto está mal.

9. Un gasto recurrente puede quedar marcado "pagado" dos veces (o nunca) según la periodicidad de la cuenta

`lib/recurringPayments.ts`

Si ya pagaste algo se decide comparando contra `profile.budgetPeriod` (quincenal/semanal/mensual, la configuración general del usuario) — nunca contra la frecuencia propia del gasto (`template.frequency`).

Escenario reproducible con la configuración por defecto: cuenta quincenal + un gasto **mensual** como "NETFLIX". Lo pagas el día 10 (quincena 1) → aparece "Pagado". El día 16 cambia a quincena 2 y NETFLIX vuelve a "Pendiente" aunque ya esté pagado ese mes — si lo vuelves a pagar, queda duplicado. Al revés: un gasto **semanal** bajo periodicidad mensual se queda "pagado" todo el mes tras el primer pago, y nunca vuelve a recordarte la semana siguiente.

10. Dos gastos recurrentes con el mismo nombre se fusionan y uno desaparece

`lib/storage.ts` (`getRecurringTemplates`)

La deduplicación de plantillas recurrentes usa solo el nombre en minúsculas como llave — sin categoría ni id — así que si dos gastos distintos comparten nombre, solo sobrevive el más reciente en la lista.

Escenario: tienes "Seguro" del carro (Transporte) desde junio, y en julio empiezas a trackear otro "Seguro" de salud por separado. Desde julio, el seguro del carro desaparece por completo de Gastos Recurrentes/Categorías — sin error, sin aviso, solo se esfumó. Además, "Deshacer" borra todos los gastos que coincidan en nombre+quincena, pudiendo borrar el del seguro equivocado.

11. Guardar dos veces rápido (doble tap) puede perder o duplicar datos

[lib/storage.ts](#) (patrón general) · [CardFormModal.tsx](#) · [index.tsx](#) (Finando)

Ningún botón de "Guardar" se desactiva mientras la operación está en curso, y el guardado en AsyncStorage es "leer todo → modificar → escribir todo", sin ningún tipo de bloqueo. Dos toques casi simultáneos pueden leer el mismo estado "antes" y el segundo guardado pisa al primero.

Escenario: doble tap accidental en "Guardar" (más probable en el celular de gama baja que ya se documentó en este proyecto) al crear una tarjeta, registrar un gasto, o confirmar en Finando IA — uno de los dos registros se pierde silenciosamente, o se agenda una notificación duplicada.

Prioridad Media — vale la pena, no bloquea el lanzamiento

Bugs reales y reproducibles, pero de menor impacto o alcance más acotado.

Préstamo sin emoji muestra "Débito" en el panel de acción — [tarjetas.tsx](#) — confirmado en vivo en la tablet (screenshot disponible). Cualquier préstamo sin emoji elegido muestra la etiqueta equivocada.

Eliminar una tarjeta deja sus gastos huérfanos — [tarjetas.tsx](#) · [storage.ts](#) — los gastos con esa cardId no se reasignan ni se avisan, y siguen contando en totales generales aunque ya no aparezcan en el historial de esa tarjeta.

Editar el monto de un préstamo hacia arriba rompe "Deuda inicial" vs "Pendiente" — [CardFormModal.tsx](#) — initialBalance no se actualiza, puede terminar mostrando "Pendiente" mayor que "Deuda inicial".

Cambiar el tipo de cuenta antes de guardar deja campos basura — [CardFormModal.tsx](#) — ej. pasar de Débito a Efectivo conserva un "Banco" ya tipeado que Efectivo no usa ni muestra.

Sin tope de caracteres en ningún campo de monto — [tarjetas.tsx](#) · [CardFormModal.tsx](#) — confirmado en vivo: se pudo guardar \$999.999.999.999.999 sin ningún aviso.

Editar un abono ya existente no valida contra la deuda pendiente — [tarjetas.tsx](#) — a diferencia de crear un abono nuevo (que sí avisa si te pasas), editar uno viejo no tiene ese control.

Editar un gasto desde el historial de Tarjetas no valida contra el saldo/cupo — [tarjetas.tsx](#) — el mismo control que sí existe en el popup "Gastos del mes" de Resumen falta acá.

Pagar un gasto recurrente con el botón "Pagar" borra su fecha de vencimiento — [recurringPayments.ts](#) — el nuevo registro no copia recurrenceDueDate, así que el "vence el..." desaparece tras el primer pago.

Sin validación de nombres duplicados en categorías — [CategoryFormModal.tsx](#) — se puede crear dos veces "Comida" con colores distintos, confuso al elegir categoría.

Eliminar una categoría deja gastos recurrentes huérfanos + colisión de key en React — [categorias.tsx](#) · [gastos-recurrentes.tsx](#) — el gasto sigue generándose contra una categoría que ya no existe, agrupado en "Sin categoría"; si pasa con dos categorías distintas, ambos grupos comparten la misma key y pueden mezclarse visualmente.

Crear un gasto recurrente desde el botón "+" de Gastos Recurrentes nunca agenda notificación — [gastos-recurrentes.tsx](#) — a diferencia de crearlo desde Categorías, este camino no llama a `scheduleRecurringReminder`.

Renombrar un gasto recurrente deja la notificación con el nombre viejo — [CategoryDetailModal.tsx](#) — solo se reprograma si cambia si-es-recurrente o la frecuencia, no si solo cambia el nombre.

En el selector de tarjeta de Finando IA, doble tap en dos cuentas distintas puede duplicar o perder la asignación — [index.tsx](#)

Una sesión expirada se trata igual que "modo anónimo", sin avisar — [lib/claude.ts](#) — el usuario sigue chateando creyendo que habla con la IA real.

Guardia débil contra doble envío en Finando IA — [index.tsx](#) — IDs basados solo en `Date.now()`, dos taps casi simultáneos pueden colisionar.

Cerrar sesión no cancela la notificación fija de saldo — [perfil.tsx](#) — si estaba activada, la notificación sigue mostrando el saldo real después de cerrar sesión.

La pantalla de Ayuda (y su PDF exportable) describen una versión vieja de Resumen — [ayuda.tsx](#) — menciona una "barra de ahorro" y solo 3 diagramas; hoy son 4 donuts distintos, sin barra.

El presupuesto mensual no se traslada de un mes a otro — [personalizacion.tsx](#) · [storage.ts](#) — cada mes nuevo empieza en "Sin configurar" aunque lo hayas configurado todos los meses anteriores.

El recordatorio mensual no está ajustado para meses cortos — [notifications.ts](#) — un recurrente marcado el día 31 puede saltarse silenciosamente febrero, abril, etc.

Posible condición de carrera en la migración de datos antiguos al arrancar — [storage.ts](#) — ventana teórica donde categorías legacy podrían perderse si varias pantallas cargan a la vez; baja probabilidad hoy, pero late en el código.

Botón "Guardar" de tarjeta/cuenta no da ningún aviso si falta el monto — [CardFormModal.tsx](#) — confirmado en vivo: solo "Cancelar" muestra el diálogo "Falta el monto"; tocar "Guardar" directamente no hace nada y no explica por qué.

Prioridad Baja — cosmético o muy poco probable

Bajar el límite de una tarjeta de crédito por debajo de lo ya gastado no avisa — [CardFormModal.tsx](#) — "Disponible" queda en \$0 sin explicación.

El campo de emoji personalizado puede truncar emojis compuestos — [CardFormModal.tsx](#) (`maxLength=2` cuenta unidades UTF-16, no emojis completos).

Doble navegación redundante al cerrar sesión — [perfil.tsx](#) — no rompe nada, pero dispara `router.replace` dos veces seguidas.

Sin tope superior en presupuesto mensual / gasto hormiga — [BudgetFormModal.tsx](#) — un cero de más (ej. \$300.000.000 en vez de \$30.000) etiqueta todo el mes como "hormiga".

La pantalla de Ayuda se contradice sobre si hace falta una API key — [ayuda.tsx](#)

formatCOP pone el símbolo "\$" antes del signo negativo — [expenseParser.ts](#) — un total negativo se ve "\$-50.000" en vez de "-\$50.000".

Historial recalcula totales de todos los meses en cada render, sin virtualizar la lista — [history.tsx](#) — con un año+ de historial puede sentirse lento al expandir/colapsar, sobre todo en Android de gama baja.



Limitación conocida — no es un bug de código

Números cortados en la leyenda de los diagramas de torta (solo en una tablet específica)

Reportado el 27 de julio. Se investigó a fondo con la tablet conectada en vivo: se probaron 3 estrategias de layout completamente distintas (incluyendo quitar el `marginLeft: 'auto'` a pedido tuyo) y, como prueba definitiva, se le dio al monto **150px de ancho garantizado** — casi 3 veces más de lo que necesita — y el corte se reprodujo exactamente igual.

Esto descarta layout por completo: el texto se recorta en el pintado del glyph (motor gráfico Skia de esa tablet específica, Android 10 de gama baja), no en el cálculo del tamaño de la caja. No hay ningún ajuste de estilo en React Native que pueda arreglar esto desde el código de la app.

Qué hacer: nada por ahora. Si aparece en el celular de algún amigo durante las pruebas, avísame — ahí sí sería un patrón real y no un caso aislado de este hardware puntual.

Auditoría generada combinando PECAS (arquitectura), ESTETIK (criterios de UX) y AUDITA (bugs) — 4 revisiones de código en paralelo sobre la totalidad del proyecto, cruzadas contra el historial de NOTAS/ajustes `wallet-control.txt` para no repetir nada ya resuelto, más pruebas en vivo en un dispositivo Android real conectado por ADB.